

Zoho Round 2 Preparation Guide

50 Curated Previous Year Questions & Java Solutions

Optimized for Campus Placements and Algorithmic Evaluations

1. Print String in Cross Pattern

Print a string of odd length in an X pattern. Often asked in Zoho.

Sample I/O:

Input: s = 'abcde'

Output:

```
a  e
b  d
 c
b  d
a  e
```

```
class Solution {
    public void printCross(String s) {
        int n = s.length();
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j || i + j == n - 1) System.out.print(s.charAt(j));
                else System.out.print(" ");
            }
            System.out.println();
        }
    }
}
```

2. Run Length Decoding

Decode a string like 'a1b10' to 'abbbbbbbbbb'.

Sample I/O:

Input: s = 'a1b10'

Output: 'abbbbbbbbbb'

```

class Solution {
    public void decode(String s) {
        for (int i = 0; i < s.length(); ) {
            char c = s.charAt(i++);
            int count = 0;
            while (i < s.length() && Character.isDigit(s.charAt(i))) {
                count = count * 10 + (s.charAt(i) - '0');
                i++;
            }
            for (int j = 0; j < count; j++) System.out.print(c);
        }
    }
}

```

3. Sort by Frequency

Sort an array based on element frequency. If frequencies match, preserve order or value.

Sample I/O:

Input: nums = [1,1,2,2,2,3]

Output: [3,1,1,2,2,2]

```

import java.util.*;
class Solution {
    public int[] frequencySort(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int n : nums) map.put(n, map.getOrDefault(n, 0) + 1);
        List<Integer> list = new ArrayList<>();
        for (int n : nums) list.add(n);
        list.sort((a, b) -> map.get(a).equals(map.get(b)) ? b - a : map.get(b) -
map.get(a));
        return list.stream().mapToInt(i -> i).toArray();
    }
}

```

4. Next Greater Element

Find the next greater element for each element in an array.

Sample I/O:

Input: nums = [1,2,1]

Output: [2,-1,2]

```
import java.util.*;
class Solution {
    public int[] nextGreaterElements(int[] nums) {
        int[] res = new int[nums.length];
        Arrays.fill(res, -1);
        Stack<Integer> stack = new Stack<>();
        for (int i = 0; i < nums.length; i++) {
            while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
                res[stack.pop()] = nums[i];
            }
            stack.push(i);
        }
        return res;
    }
}
```

5. Rotate Matrix 90 Degrees

Rotate an $N \times N$ matrix 90 degrees clockwise in place.

Sample I/O:

Input: matrix = [[1,2],[3,4]]

Output: [[3,1],[4,2]]

```
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n / 2; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[i][n - 1 - j];
                matrix[i][n - 1 - j] = temp;
            }
        }
    }
}
```

6. Spiral Matrix

Return all elements of a matrix in spiral order.

Sample I/O:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [1,2,3,6,9,8,7,4,5]

```

import java.util.*;
class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> res = new ArrayList<>();
        if (matrix.length == 0) return res;
        int top = 0, bottom = matrix.length - 1, left = 0, right = matrix[0].length - 1;
        while (top <= bottom && left <= right) {
            for (int i = left; i <= right; i++) res.add(matrix[top][i]);
            top++;
            for (int i = top; i <= bottom; i++) res.add(matrix[i][right]);
            right--;
            if (top <= bottom) {
                for (int i = right; i >= left; i--) res.add(matrix[bottom][i]);
                bottom--;
            }
            if (left <= right) {
                for (int i = bottom; i >= top; i--) res.add(matrix[i][left]);
                left++;
            }
        }
        return res;
    }
}

```

7. Remove Unbalanced Parentheses

Remove minimum number of parentheses to make string balanced.

Sample I/O:

Input: s = 'lee(t(c)o)de'

Output: 'lee(t(c)o)de'

```

class Solution {
    public String minRemoveToMakeValid(String s) {
        StringBuilder sb = new StringBuilder(s);
        int open = 0;
        for (int i = 0; i < sb.length(); i++) {
            if (sb.charAt(i) == '(') open++;
            else if (sb.charAt(i) == ')') {
                if (open == 0) { sb.deleteCharAt(i--); }
                else open--;
            }
        }
        for (int i = sb.length() - 1; i >= 0 && open > 0; i--) {
            if (sb.charAt(i) == '(') {
                sb.deleteCharAt(i);
                open--;
            }
        }
        return sb.toString();
    }
}

```

8. Reverse Words in String

Reverse the order of words in a string.

Sample I/O:

Input: s = 'the sky is blue'

Output: 'blue is sky the'

```

class Solution {
    public String reverseWords(String s) {
        String[] words = s.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();
        for (int i = words.length - 1; i >= 0; i--) {
            sb.append(words[i]);
            if (i > 0) sb.append(" ");
        }
        return sb.toString();
    }
}

```

9. Missing Number

Find the missing number in an array containing n distinct numbers taken from 0, 1, 2, ..., n.

Sample I/O:

Input: nums = [3,0,1]

Output: 2

```

class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;
        for (int num : nums) actualSum += num;
        return expectedSum - actualSum;
    }
}

```

10. Two Sum

Given an array of integers and a target, return indices of the two numbers that add up to target.

Sample I/O:

Input: nums = [2,7,11,15], target = 9

Output: [0,1]

```

import java.util.*;
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[] { map.get(complement), i };
            }
            map.put(nums[i], i);
        }
        return new int[]{};
    }
}

```

11. Valid Palindrome

Determine if a string is a valid palindrome, ignoring non-alphanumeric characters.

Sample I/O:

Input: s = 'A man, a plan, a canal: Panama'

Output: true

```

class Solution {
    public boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
        while (left < right) {
            while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
            while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
            if (Character.toLowerCase(s.charAt(left)) !=
                Character.toLowerCase(s.charAt(right))) return false;
            left++; right--;
        }
        return true;
    }
}

```

12. Move Zeroes

Move all 0's to the end of the array while maintaining the relative order of non-zero elements.

Sample I/O:

Input: nums = [0,1,0,3,12]

Output: [1,3,12,0,0]

```

class Solution {
    public void moveZeroes(int[] nums) {
        int insertPos = 0;
        for (int num : nums) {
            if (num != 0) nums[insertPos++] = num;
        }
        while (insertPos < nums.length) {
            nums[insertPos++] = 0;
        }
    }
}

```

13. Remove Duplicates

Remove duplicates from a sorted array in-place.

Sample I/O:

Input: nums = [1,1,2]

Output: 2 (array becomes [1,2,_])

```

class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;
        int i = 0;
        for (int j = 1; j < nums.length; j++) {
            if (nums[j] != nums[i]) {
                i++;
                nums[i] = nums[j];
            }
        }
        return i + 1;
    }
}

```

14. Majority Element

Find the majority element that appears more than $n/2$ times.

Sample I/O:

Input: nums = [3,2,3]

Output: 3

```

class Solution {
    public int majorityElement(int[] nums) {
        int count = 0, candidate = 0;
        for (int num : nums) {
            if (count == 0) candidate = num;
            count += (num == candidate) ? 1 : -1;
        }
        return candidate;
    }
}

```

15. Maximum Subarray (Kadane)

Find the contiguous subarray with the largest sum.

Sample I/O:

Input: nums = [-2,1,-3,4,-1,2,1,-5,4]

Output: 6


```

class Solution {
    public int maxSubArray(int[] nums) {
        int maxSum = nums[0], currentSum = nums[0];
        for (int i = 1; i < nums.length; i++) {
            currentSum = Math.max(nums[i], currentSum + nums[i]);
            maxSum = Math.max(maxSum, currentSum);
        }
        return maxSum;
    }
}

```

16. Merge Intervals

Merge all overlapping intervals.

Sample I/O:

Input: intervals = [[1,3],[2,6],[8,10],[15,18]]

Output: [[1,6],[8,10],[15,18]]

```

import java.util.*;
class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length <= 1) return intervals;
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        List<int[]> res = new ArrayList<>();
        int[] current = intervals[0];
        res.add(current);
        for (int[] interval : intervals) {
            if (current[1] >= interval[0]) {
                current[1] = Math.max(current[1], interval[1]);
            } else {
                current = interval;
                res.add(current);
            }
        }
        return res.toArray(new int[res.size()][]);
    }
}

```

17. Group Anagrams

Group anagrams together from an array of strings.

Sample I/O:

Input: strs = ['eat','tea','tan','ate','nat','bat']

Output: [['bat'],['nat','tan'],['ate','eat','tea']]

```
import java.util.*;
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            char[] chars = s.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

18. Valid Anagram

Check if two strings are anagrams of each other.

Sample I/O:

Input: s = 'anagram', t = 'nagaram'

Output: true

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] count = new int[26];
        for (int i = 0; i < s.length(); i++) {
            count[s.charAt(i) - 'a']++;
            count[t.charAt(i) - 'a']--;
        }
        for (int c : count) {
            if (c != 0) return false;
        }
        return true;
    }
}
```

19. Longest Common Prefix

Find the longest common prefix string amongst an array of strings.

Sample I/O:

Input: strs = ['flower', 'flow', 'flight']

Output: 'fl'

```

class Solution {
    public String longestCommonPrefix(String[] strs) {
        if (strs.length == 0) return "";
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (strs[i].indexOf(prefix) != 0) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
}

```

20. Trapping Rain Water

Calculate how much water it can trap after raining.

Sample I/O:

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

```

class Solution {
    public int trap(int[] height) {
        int left = 0, right = height.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;
        while (left < right) {
            if (height[left] < height[right]) {
                if (height[left] >= leftMax) leftMax = height[left];
                else water += leftMax - height[left];
                left++;
            } else {
                if (height[right] >= rightMax) rightMax = height[right];
                else water += rightMax - height[right];
                right--;
            }
        }
        return water;
    }
}

```

21. Container with Most Water

Find two lines that together with the x-axis form a container that holds the most water.

Sample I/O:

Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

```

class Solution {
    public int maxArea(int[] height) {
        int left = 0, right = height.length - 1, maxArea = 0;
        while (left < right) {
            int currentArea = Math.min(height[left], height[right]) * (right - left);
            maxArea = Math.max(maxArea, currentArea);
            if (height[left] < height[right]) left++;
            else right--;
        }
        return maxArea;
    }
}

```

22. Search Insert Position

Return the index if the target is found, or where it would be if it were inserted in order.

Sample I/O:

Input: nums = [1,3,5,6], target = 5

Output: 2

```

class Solution {
    public int searchInsert(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) return mid;
            else if (nums[mid] < target) left = mid + 1;
            else right = mid - 1;
        }
        return left;
    }
}

```

23. First and Last Position in Sorted Array

Find the starting and ending position of a given target value.

Sample I/O:

Input: nums = [5,7,7,8,8,10], target = 8

Output: [3,4]

```

class Solution {
    public int[] searchRange(int[] nums, int target) {
        int[] result = {-1, -1};
        result[0] = findBound(nums, target, true);
        result[1] = findBound(nums, target, false);
        return result;
    }
    private int findBound(int[] nums, int target, boolean isFirst) {
        int left = 0, right = nums.length - 1, bound = -1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                bound = mid;
                if (isFirst) right = mid - 1;
                else left = mid + 1;
            } else if (nums[mid] < target) left = mid + 1;
            else right = mid - 1;
        }
        return bound;
    }
}

```

24. Length of Last Word

Given a string, return the length of the last word in the string.

Sample I/O:
 Input: s = 'Hello World'
 Output: 5

```

class Solution {
    public int lengthOfLastWord(String s) {
        int length = 0, i = s.length() - 1;
        while (i >= 0 && s.charAt(i) == ' ') i--;
        while (i >= 0 && s.charAt(i) != ' ') {
            length++;
            i--;
        }
        return length;
    }
}

```

25. Multiply Strings

Multiply two non-negative integers represented as strings.

Sample I/O:
 Input: num1 = '2', num2 = '3'
 Output: '6'

```

class Solution {
    public String multiply(String num1, String num2) {
        if (num1.equals("0") || num2.equals("0")) return "0";
        int[] res = new int[num1.length() + num2.length()];
        for (int i = num1.length() - 1; i >= 0; i--) {
            for (int j = num2.length() - 1; j >= 0; j--) {
                int mul = (num1.charAt(i) - '0') * (num2.charAt(j) - '0');
                int sum = mul + res[i + j + 1];
                res[i + j + 1] = sum % 10;
                res[i + j] += sum / 10;
            }
        }
        StringBuilder sb = new StringBuilder();
        for (int p : res) if (!(sb.length() == 0 && p == 0)) sb.append(p);
        return sb.toString();
    }
}

```

26. Add Binary

Add two binary strings and return their sum as a binary string.

Sample I/O:

Input: a = '11', b = '1'

Output: '100'

```

class Solution {
    public String addBinary(String a, String b) {
        StringBuilder sb = new StringBuilder();
        int i = a.length() - 1, j = b.length() - 1, carry = 0;
        while (i >= 0 || j >= 0) {
            int sum = carry;
            if (i >= 0) sum += a.charAt(i--) - '0';
            if (j >= 0) sum += b.charAt(j--) - '0';
            sb.append(sum % 2);
            carry = sum / 2;
        }
        if (carry != 0) sb.append(carry);
        return sb.reverse().toString();
    }
}

```

27. Climbing Stairs

Find distinct ways to climb n stairs taking 1 or 2 steps.

Sample I/O:

Input: n = 2

Output: 2

```

class Solution {
    public int climbStairs(int n) {
        if (n <= 2) return n;
        int a = 1, b = 2;
        for (int i = 3; i <= n; i++) {
            int temp = a + b;
            a = b;
            b = temp;
        }
        return b;
    }
}

```

28. Sqrt(x)

Compute and return the square root of x , truncated to an integer.

Sample I/O:

Input: $x = 4$

Output: 2

```

class Solution {
    public int mySqrt(int x) {
        if (x == 0) return 0;
        int left = 1, right = x, ans = 0;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (mid <= x / mid) {
                ans = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return ans;
    }
}

```

29. Pascal's Triangle

Generate the first $numRows$ of Pascal's triangle.

Sample I/O:

Input: $numRows = 5$

Output: $[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]$

```
import java.util.*;
class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> triangle = new ArrayList<>();
        if (numRows == 0) return triangle;
        triangle.add(new ArrayList<>());
        triangle.get(0).add(1);
        for (int i = 1; i < numRows; i++) {
            List<Integer> row = new ArrayList<>();
            List<Integer> prevRow = triangle.get(i - 1);
            row.add(1);
            for (int j = 1; j < i; j++) {
                row.add(prevRow.get(j - 1) + prevRow.get(j));
            }
            row.add(1);
            triangle.add(row);
        }
        return triangle;
    }
}
```

30. Best Time to Buy and Sell Stock

Maximize profit by choosing a single day to buy and a different day to sell.

Sample I/O:

Input: prices = [7,1,5,3,6,4]

Output: 5

```
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;
        for (int price : prices) {
            if (price < minPrice) minPrice = price;
            else if (price - minPrice > maxProfit) maxProfit = price - minPrice;
        }
        return maxProfit;
    }
}
```

31. Valid Sudoku

Determine if a 9 x 9 Sudoku board is valid.

Sample I/O:

Input: board = [...] (9x9 Sudoku)

Output: true


```

import java.util.*;
class Solution {
    public boolean isValidSudoku(char[][] board) {
        Set<String> seen = new HashSet<>();
        for (int i=0; i<9; i++) {
            for (int j=0; j<9; j++) {
                char c = board[i][j];
                if (c != '.') {
                    if (!seen.add(c + " in row " + i) ||
                        !seen.add(c + " in col " + j) ||
                        !seen.add(c + " in block " + i/3 + "-" + j/3))
                        return false;
                }
            }
        }
        return true;
    }
}

```

32. Subarray Sum Equals K

Find the total number of continuous subarrays whose sum equals to k.

Sample I/O:

Input: nums = [1,1,1], k = 2

Output: 2

```

import java.util.*;
class Solution {
    public int subarraySum(int[] nums, int k) {
        int count = 0, sum = 0;
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
            if (map.containsKey(sum - k)) count += map.get(sum - k);
            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }
        return count;
    }
}

```

33. First Missing Positive

Find the smallest missing positive integer in $O(n)$ time.

Sample I/O:

Input: nums = [1,2,0]

Output: 3

```

class Solution {
    public int firstMissingPositive(int[] nums) {
        int i = 0;
        while (i < nums.length) {
            if (nums[i] > 0 && nums[i] <= nums.length && nums[nums[i] - 1] != nums[i]) {
                int temp = nums[nums[i] - 1];
                nums[nums[i] - 1] = nums[i];
                nums[i] = temp;
            } else i++;
        }
        for (i = 0; i < nums.length; i++) {
            if (nums[i] != i + 1) return i + 1;
        }
        return nums.length + 1;
    }
}

```

34. Set Matrix Zeroes

If an element is 0, set its entire row and column to 0 in-place.

Sample I/O:

Input: matrix = [[1,1,1],[1,0,1],[1,1,1]]

Output: [[1,0,1],[0,0,0],[1,0,1]]

```

class Solution {
    public void setZeroes(int[][] matrix) {
        boolean firstRowZero = false, firstColZero = false;
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[0].length; j++) {
                if (matrix[i][j] == 0) {
                    if (i == 0) firstRowZero = true;
                    if (j == 0) firstColZero = true;
                    matrix[0][j] = 0;
                    matrix[i][0] = 0;
                }
            }
        }
        for (int i = 1; i < matrix.length; i++) {
            for (int j = 1; j < matrix[0].length; j++) {
                if (matrix[i][0] == 0 || matrix[0][j] == 0) matrix[i][j] = 0;
            }
        }
        if (firstRowZero) for (int j = 0; j < matrix[0].length; j++) matrix[0][j] = 0;
        if (firstColZero) for (int i = 0; i < matrix.length; i++) matrix[i][0] = 0;
    }
}

```

35. Find Duplicate Number

Find the repeated number in an array without modifying it.

Sample I/O:

Input: nums = [1,3,4,2,2]

Output: 2

```
class Solution {
    public int findDuplicate(int[] nums) {
        int slow = nums[0], fast = nums[nums[0]];
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[nums[fast]];
        }
        fast = 0;
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;
    }
}
```

36. Minimum Path Sum

*Find a path from top-left to bottom-right minimizing the sum.***Sample I/O:**

Input: grid = [[1,3,1],[1,5,1],[4,2,1]]

Output: 7

```
class Solution {
    public int minPathSum(int[][] grid) {
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[0].length; j++) {
                if (i == 0 && j == 0) continue;
                else if (i == 0) grid[i][j] += grid[i][j - 1];
                else if (j == 0) grid[i][j] += grid[i - 1][j];
                else grid[i][j] += Math.min(grid[i - 1][j], grid[i][j - 1]);
            }
        }
        return grid[grid.length - 1][grid[0].length - 1];
    }
}
```

37. Word Search

*Check if a word exists in a grid of characters.***Sample I/O:**

Input: board = [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word = 'ABCCED'

Output: true

```

class Solution {
    public boolean exist(char[][] board, String word) {
        for (int i = 0; i < board.length; i++)
            for (int j = 0; j < board[0].length; j++)
                if (dfs(board, i, j, word, 0)) return true;
        return false;
    }
    private boolean dfs(char[][] b, int i, int j, String w, int idx) {
        if (idx == w.length()) return true;
        if (i < 0 || i >= b.length || j < 0 || j >= b[0].length || b[i][j] != w.charAt(idx))
            return false;
        char temp = b[i][j];
        b[i][j] = ' ';
        boolean found = dfs(b, i+1, j, w, idx+1) || dfs(b, i-1, j, w, idx+1) ||
            dfs(b, i, j+1, w, idx+1) || dfs(b, i, j-1, w, idx+1);
        b[i][j] = temp;
        return found;
    }
}

```

38. Subsets

Return all possible subsets (the power set) of an array.

Sample I/O:

Input: nums = [1,2,3]

Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

```

import java.util.*;
class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        generate(0, nums, new ArrayList<>(), res);
        return res;
    }
    private void generate(int index, int[] nums, List<Integer> current,
        List<List<Integer>> res) {
        res.add(new ArrayList<>(current));
        for (int i = index; i < nums.length; i++) {
            current.add(nums[i]);
            generate(i + 1, nums, current, res);
            current.remove(current.size() - 1);
        }
    }
}

```

39. Permutations

Return all possible permutations of an array.

Sample I/O:

Input: nums = [1,2,3]

Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

```
import java.util.*;
class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(res, new ArrayList<>(), nums);
        return res;
    }
    private void backtrack(List<List<Integer>> res, List<Integer> curr, int[] nums) {
        if (curr.size() == nums.length) {
            res.add(new ArrayList<>(curr));
            return;
        }
        for (int i = 0; i < nums.length; i++) {
            if (curr.contains(nums[i])) continue;
            curr.add(nums[i]);
            backtrack(res, curr, nums);
            curr.remove(curr.size() - 1);
        }
    }
}
```

40. Combinations

*Return all combinations of k numbers out of 1 to n.***Sample I/O:**

Input: n = 4, k = 2

Output: [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]

```

import java.util.*;
class Solution {
    public List<List<Integer>> combine(int n, int k) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(1, new ArrayList<>(), n, k, res);
        return res;
    }
    private void backtrack(int start, List<Integer> curr, int n, int k,
List<List<Integer>> res) {
        if (curr.size() == k) {
            res.add(new ArrayList<>(curr));
            return;
        }
        for (int i = start; i <= n; i++) {
            curr.add(i);
            backtrack(i + 1, curr, n, k, res);
            curr.remove(curr.size() - 1);
        }
    }
}

```

41. Letter Combinations of a Phone Number

Return all letter combinations a digit string could represent.

Sample I/O:

Input: digits = '23'

Output: ['ad', 'ae', 'af', 'bd', 'be', 'bf', 'cd', 'ce', 'cf']

```

import java.util.*;
class Solution {
    public List<String> letterCombinations(String digits) {
        LinkedList<String> ans = new LinkedList<>();
        if (digits.isEmpty()) return ans;
        String[] mapping = {"0", "1", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv",
"xyz"};
        ans.add("");
        for (int i = 0; i < digits.length(); i++) {
            int x = Character.getNumericValue(digits.charAt(i));
            while (ans.peek().length() == i) {
                String t = ans.remove();
                for (char s : mapping[x].toCharArray())
                    ans.add(t + s);
            }
        }
        return ans;
    }
}

```

42. Generate Parentheses

Generate all combinations of well-formed parentheses.

Sample I/O:

Input: n = 3

Output: ['((()))', '(()())', '(()())', '()()()', '()()()']

```
import java.util.*;
class Solution {
    public List<String> generateParenthesis(int n) {
        List<String> res = new ArrayList<>();
        backtrack(res, "", 0, 0, n);
        return res;
    }
    private void backtrack(List<String> res, String str, int open, int close, int max) {
        if (str.length() == max * 2) {
            res.add(str);
            return;
        }
        if (open < max) backtrack(res, str + "(", open + 1, close, max);
        if (close < open) backtrack(res, str + ")", open, close + 1, max);
    }
}
```

43. Search in Rotated Sorted Array

Search for target in a rotated sorted array.

Sample I/O:

Input: nums = [4,5,6,7,0,1,2], target = 0

Output: 4

```
class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) return mid;
            if (nums[left] <= nums[mid]) {
                if (target >= nums[left] && target < nums[mid]) right = mid - 1;
                else left = mid + 1;
            } else {
                if (target > nums[mid] && target <= nums[right]) left = mid + 1;
                else right = mid - 1;
            }
        }
        return -1;
    }
}
```

44. Find Minimum in Rotated Sorted Array

Find the minimum element in a rotated sorted array.

Sample I/O:

Input: nums = [3,4,5,1,2]

Output: 1

```
class Solution {
    public int findMin(int[] nums) {
        int left = 0, right = nums.length - 1;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] > nums[right]) left = mid + 1;
            else right = mid;
        }
        return nums[left];
    }
}
```

45. Longest Palindromic Substring

Find the longest palindromic substring.

Sample I/O:

Input: s = 'babad'

Output: 'bab'

```
class Solution {
    public String longestPalindrome(String s) {
        if (s == null || s.length() < 1) return "";
        int start = 0, end = 0;
        for (int i = 0; i < s.length(); i++) {
            int len1 = expandAroundCenter(s, i, i);
            int len2 = expandAroundCenter(s, i, i + 1);
            int len = Math.max(len1, len2);
            if (len > end - start) {
                start = i - (len - 1) / 2;
                end = i + len / 2;
            }
        }
        return s.substring(start, end + 1);
    }
    private int expandAroundCenter(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            left--; right++;
        }
        return right - left - 1;
    }
}
```


46. Longest Substring Without Repeating

Find the length of the longest substring without repeating characters.

Sample I/O:

Input: s = 'abcabcbb'

Output: 3

```
import java.util.*;
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int n = s.length(), ans = 0;
        Map<Character, Integer> map = new HashMap<>();
        for (int j = 0, i = 0; j < n; j++) {
            if (map.containsKey(s.charAt(j))) {
                i = Math.max(map.get(s.charAt(j)), i);
            }
            ans = Math.max(ans, j - i + 1);
            map.put(s.charAt(j), j + 1);
        }
        return ans;
    }
}
```

47. Zigzag Conversion

Convert a string into a zigzag pattern on a given number of rows.

Sample I/O:

Input: s = 'PAYPALISHIRING', numRows = 3

Output: 'PAHNAPLSIIGYIR'

```
class Solution {
    public String convert(String s, int numRows) {
        if (numRows == 1) return s;
        StringBuilder[] rows = new StringBuilder[numRows];
        for (int i = 0; i < numRows; i++) rows[i] = new StringBuilder();
        int curRow = 0;
        boolean goingDown = false;
        for (char c : s.toCharArray()) {
            rows[curRow].append(c);
            if (curRow == 0 || curRow == numRows - 1) goingDown = !goingDown;
            curRow += goingDown ? 1 : -1;
        }
        StringBuilder res = new StringBuilder();
        for (StringBuilder row : rows) res.append(row);
        return res.toString();
    }
}
```

48. Reverse Integer

Reverse the digits of a 32-bit signed integer.

Sample I/O:

Input: x = 123

Output: 321

```
class Solution {
    public int reverse(int x) {
        long res = 0;
        while (x != 0) {
            res = res * 10 + x % 10;
            x /= 10;
        }
        if (res < Integer.MIN_VALUE || res > Integer.MAX_VALUE) return 0;
        return (int) res;
    }
}
```

49. String to Integer (atoi)

Convert a string to a 32-bit signed integer.

Sample I/O:

Input: s = ' -42'

Output: -42

```
class Solution {
    public int myAtoi(String str) {
        int index = 0, sign = 1, total = 0;
        if (str.length() == 0) return 0;
        while (index < str.length() && str.charAt(index) == ' ') index++;
        if (index < str.length() && (str.charAt(index) == '+' || str.charAt(index) == '-')) {
            sign = str.charAt(index) == '+' ? 1 : -1;
            index++;
        }
        while (index < str.length() && Character.isDigit(str.charAt(index))) {
            int digit = str.charAt(index) - '0';
            if (Integer.MAX_VALUE / 10 < total || (Integer.MAX_VALUE / 10 == total && Integer.MAX_VALUE % 10 < digit))
                return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
            total = total * 10 + digit;
            index++;
        }
        return total * sign;
    }
}
```

50. Palindrome Number

Check if an integer is a palindrome without converting it to string.

Sample I/O:

Input: x = 121

Output: true

```
class Solution {
    public boolean isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false;
        int revertedNumber = 0;
        while (x > revertedNumber) {
            revertedNumber = revertedNumber * 10 + x % 10;
            x /= 10;
        }
        return x == revertedNumber || x == revertedNumber / 10;
    }
}
```